Docker allows us to create a few different types of volumes, all of which would do the job. For example, we've already seen how to mount a local volume. However, there's another type of volume better suited to our purpose. We don't really care *where* or *how* the files are stored, we just care that they *are* stored somewhere separately. For this, we can create a *named volume*: a self-contained *bucket* of file storage, completely managed by Docker.

But enough of the theory; let's see how we do this in practice.

Named volumes can be created and managed through the `docker volume` command. While that's worth knowing about, since we're using Compose, we can let it handle the management of the volumes for us.

Here's our `docker-compose.yml` modified to store our persistent data on a volume:

```
Line 1  version: '3'
-
-       services:
-
5       web:
-         build: .
-         ports:
-           - "3000:3000"
-         volumes:
10          - .:/usr/src/app
-         env_file:
-           - .env/development/database
-           - .env/development/web
-
15      redis:
-         image: redis
-
-       database:
-         image: postgres
20        env_file:
-           - .env/development/database
-         volumes:
-           - db_data:/var/lib/postgresql/data
-
25      volumes:
-       db_data:
```

The first step is to tell Compose that we need a named volume. Named volumes are defined under the top-level `volumes` property (line 25); here, we've defined a named volume called `db_data` (line 26).

Next, we need to tell Compose to mount the named volume inside our database container, using the familiar `volumes` property (line 22). Mounting a named volume (line 23) is similar to mounting a local directory (line 10)—the difference